

Pregunta 1: Forward Pass en Red Multicapa (MLP)

Objetivo: Calcular manualmente la salida de la red neuronal dado un vector de entrada.

Datos del problema:

- Entrada:** $X = [x_1, x_2]^T = [2, 1]^T$.
- Arquitectura:**
 - Entrada: 2 neuronas.
 - Oculto: 2 neuronas (h_1, h_2) con activación **ReLU** ($f(z) = \max(0, z)$).
 - Salida: 1 neurona ($\hat{y}$) con activación **Escalón Unitario** ($g(z) = 1$ si $z \geq 0$, sino 0).
- Pesos y Sesgos:**
 - $W^{(1)} = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}$ (Fila 1 para h_1 , Fila 2 para h_2).
 - $b^{(1)} = \begin{bmatrix} -1 \\ -5 \end{bmatrix}$.
 - $W^{(2)} = \begin{bmatrix} 3 & -2 \end{bmatrix}$.
 - $b^{(2)} = 1$.

Paso 1: Diagrama de la Red

Aunque no puedo dibujar a mano alzada aquí, describiré las conexiones exactas que debes dil

- Dibuja 2 nodos a la izquierda ($x_1 = 2, x_2 = 1$).
- Dibuja 2 nodos en el centro (h_1, h_2).
- Dibuja 1 nodo a la derecha ($\hat{y}$).

4. Conexiones capa Entrada $\rightarrow$ Oculta:

- De x_1 a h_1 : peso 1.
- De x_2 a h_1 : peso -1 .
- De x_1 a h_2 : peso 2.
- De x_2 a h_2 : peso 1.
- Sesgo entrando a h_1 : -1 .*
- Sesgo entrando a h_2 : -5 .*

5. Conexiones capa Oculta $\rightarrow$ Salida:

- De h_1 a $\hat{y}$: peso 3.
- De h_2 a $\hat{y}$: peso -2 .
- Sesgo entrando a $\hat{y}$: 1.*

Paso 2: Cálculo en la Capa Oculta

Calculamos la entrada neta (z) y la activación (a) para cada neurona oculta usando la fórmula $z = \sum(w \cdot x) + b$.

Para la neurona h_1 :

- Entrada neta (z_1):**

$$z_1 = (w_{11} \cdot x_1) + (w_{12} \cdot x_2) + b_1$$

$$z_1 = (1 \cdot 2) + (-1 \cdot 1) + (-1)$$

$$z_1 = 2 - 1 - 1 = 0$$

- Activación (a_1):** Usamos ReLU.

$$a_1 = \max(0, z_1) = \max(0, 0) = 0$$

Para la neurona h_2 :

- Entrada neta (z_2):**

$$z_2 = (w_{21} \cdot x_1) + (w_{22} \cdot x_2) + b_2$$

$$z_2 = (2 \cdot 2) + (1 \cdot 1) + (-5)$$

$$z_2 = 4 + 1 - 5 = 0$$

- Activación (a_2):** Usamos ReLU.

$$a_2 = \max(0, z_2) = \max(0, 0) = 0$$

Resultado intermedio: El vector de activación de la capa oculta es $a = [0, 0]^T$.

Paso 3: Cálculo en la Capa de Salida

Ahora usamos las salidas de la capa oculta (a_1, a_2) como entradas para la capa final.

- Entrada neta de salida (z_{out}):**

$$z_{out} = (w_1^{(2)} \cdot a_1) + (w_2^{(2)} \cdot a_2) + b^{(2)}$$

$$z_{out} = (3 \cdot 0) + (-2 \cdot 0) + 1$$

$$z_{out} = 0 + 0 + 1 = 1$$

- Predicción final ($\hat{y}$):** Usamos la función Escalón Unitario ($g(z)$).

$$g(z) = 1 \text{ si } z \geq 0$$

Como $z_{out} = 1$ y $1 \geq 0$:

$$\hat{y} = 1$$

Paso 4: Interpretación del Resultado

La red neuronal ha producido una salida de **1**.
Dado que se trata de un problema de clasificación binaria (Clase 0 o Clase 1):

Respuesta: La muestra pertenece a la **Clase 1**.

Pregunta 2: Funciones de Activación en Redes Neuronales

Análisis crítico de los factores de selección.

Esta pregunta pide analizar por qué elegimos ciertas funciones (como ReLU, Sigmoide, Tanh, Softmax) basándonos en matemáticas y el tipo de problema.

1. Factores para las Capas Ocultas (Hidden Layers)

En las capas intermedias, el objetivo principal es aprender representaciones complejas sin perder la capacidad de entrenar la red (Backpropagation).

- **Comportamiento del Gradiente (Vanishing Gradient):**
 - *Problema:* Funciones como la **Sigmoide** o **Tangente Hiperbólica (Tanh)** tienen derivadas que se acercan a cero en los extremos (saturación). Al multiplicar muchas derivadas pequeñas en una red profunda, el gradiente desaparece y la red deja de aprender.
 - *Solución:* La función **ReLU** ($f(x) = \max(0, x)$) es la preferida hoy en día porque su derivada es 1 para valores positivos. Esto evita el desvanecimiento del gradiente y acelera el entrenamiento.
- **Derivabilidad y Continuidad:**
 - Para usar el descenso del gradiente, las funciones deben ser derivables.
 - *Nota:* ReLU no es derivable exactamente en 0, pero matemáticamente se asume un sub-gradiente (generalmente 0 o 1), lo cual funciona perfectamente en la práctica.
- **Costo Computacional:**
 - Calcular exponenciales (como en Sigmoide/Tanh) es costoso. ReLU es simplemente una comparación (`if x > 0`), lo que es computacionalmente muy eficiente.

2. Factores para la Capa de Salida (Output Layer)

En la última capa, la elección depende estrictamente de la **naturaleza del problema** y las **propiedades estadísticas** deseadas.

Tipo de Problema	Función de Activación	Rango	Justificación
Clasificación Binaria	Sigmoide	$(0, 1)$	Queremos interpretar la salida como una probabilidad de pertenecer a la clase positiva (ej. $P(y=1)$).
Clasificación Multiclase	Softmax	$(0, 1)$	Convierte un vector de números en una distribución de probabilidad donde la suma de todas las salidas es 1. Permite elegir la clase con mayor probabilidad.
Regresión	Lineal (Identidad)	$(-\infty, \infty)$	Si queremos predecir un valor real (como el precio de una casa), no debemos restringir el rango, por lo que no se aplica no-linealidad o se usa $f(x) = x$.
Regresión (Valores +)	ReLU	$[0, \infty)$	Si la salida debe ser siempre positiva (ej. predicción de edad o tiempo), se puede usar ReLU.

Resumen para tu respuesta:

Para responder "críticamente" como pide el examen:

1. **Capas Ocultas:** Se prioriza **ReLU** sobre Sigmoide/Tanh para evitar el problema del **desvanecimiento del gradiente** y mejorar la **eficiencia computacional**.
2. **Capa de Salida:** La función se selecciona para forzar que la salida coincida con la distribución objetivo:
 - **Sigmoide** para probabilidad binaria (Bernoulli).
 - **Softmax** para distribución categórica (Multinoulli).
 - **Lineal** para regresión (Gaussiana).

Pregunta 3: Limitaciones Arquitectónicas en MLP

Objetivo: Demostrar por qué las redes "planas" antiguas fallaban al hacerse profundas y cómo se solucionó.

1. El Problema del Vanishing/Exploding Gradient (Matemáticamente)

El enunciado pide demostrar esto matemáticamente. El problema surge al aplicar la **Regla de la Cadena** durante el algoritmo de Backpropagation en una red profunda.

Imagina una red con L capas. Para actualizar los pesos de la primera capa ($W^{(1)}$), necesitamos calcular la derivada de la función de pérdida (E) respecto a ese peso.

$$\frac{\partial E}{\partial W^{(1)}} = \frac{\partial E}{\partial y} \cdot \underbrace{\frac{\partial y}{\partial h^{(L)}} \cdot \frac{\partial h^{(L)}}{\partial h^{(L-1)}} \cdots \frac{\partial h^{(2)}}{\partial h^{(1)}}}_{\text{Producto de muchas capas}} \cdot \frac{\partial h^{(1)}}{\partial W^{(1)}}$$

Cada término $\frac{\partial h^{(k)}}{\partial h^{(k-1)}}$ incluye la derivada de la función de activación, digamos sigmoide $\sigma'(z)$.

- **Vanishing (Desvanecimiento):** La derivada máxima de la sigmoide es 0.25. Si tienes 10 capas, multiplicas $0.25^{10} \approx 0.0000009$. El gradiente se vuelve casi cero, la red deja de aprender y los pesos iniciales no cambian.
- **Exploding (Explosión):** Si los pesos iniciales son grandes (ej. > 1) y no usamos sigmoide, el producto puede tender a infinito, causando inestabilidad numérica (NaN).

2. La Maldición de la Dimensionalidad

En MLPs convencionales, si aumentas la complejidad de la entrada, necesitas aumentar exponencialmente el tamaño de la capa oculta para cubrir el espacio.

- **Explicación:** En dimensiones altas, los datos se vuelven "dispersos". Para generalizar correctamente, necesitas llenar ese espacio vacío con datos de entrenamiento. Si la dimensión aumenta linealmente, la cantidad de datos necesaria para mantener la densidad estadística crece exponencialmente. Las MLPs antiguas (totalmente conectadas) sufren mucho esto porque cada neurona conecta con *todas* las anteriores, disparando el número de parámetros.

3. Capacidad de Generalización

Las MLPs tienen una alta "Capacidad" (dimensión VC).

- Si la red es muy profunda y ancha, puede aprender a memorizar el ruido de los datos de entrenamiento (Overfitting) en lugar de aprender patrones.
- Sin técnicas modernas de regularización, una MLP profunda simplemente "interpola" los datos de entrenamiento, fallando miserablemente con datos nuevos.

4. Técnicas Modernas vs. Evidencia Inicial

El enunciado pide contrastar con **ResNet** y **Normalización**, y explicar por qué no fue evidente antes.

- **ResNet (Conexiones Residuales):**
 - *Solución:* Introduce conexiones "saltadas" ($y = F(x) + x$).
 - *Por qué funciona:* Durante el backpropagation, el gradiente puede fluir a través del "+ x" (cuya derivada es 1) sin disminuir. Crea una "superautopista" para el gradiente hasta las primeras capas.
- **Normalización por Capas (Batch Norm):**
 - Fuerza a que la entrada de cada capa tenga media 0 y varianza 1. Esto evita que las neuronas entren en la zona de saturación (donde el gradiente es 0).
- **¿Por qué no eran evidentes?**
 - En los inicios del MLP, se pensaba que el problema eran los mínimos locales.
 - No existía el poder computacional para experimentar con redes de 100 capas (donde ResNet brilla).
 - Faltaba la intuición matemática de que optimizar redes profundas es más un problema de "flujo de señal" (gradientes) que de capacidad.

Pregunta 4: Concepto Teórico sobre RBM y Autoencoders

Esta pregunta toca la historia y teoría del "Deep Learning" antes de la era moderna supervisada.

1. RBM: Naturaleza y Objetivo Principal

- **Naturaleza:** Una Máquina de Boltzmann Restringida (RBM) es una red neuronal **estocástica** (aleatoria) y **generativa** de dos capas (visible y oculta). "Restringida" significa que no hay conexiones entre neuronas de la misma capa.
- **Objetivo:** Aprender la **distribución de probabilidad** de los datos de entrada sin supervisión. Busca reconstruir la entrada aprendiendo las características latentes (ocultas) que generan esos datos.

2. RBM: Función de Energía

Las RBM son modelos basados en energía (inspirados en la termodinámica).

- **Concepto:** Se define una función de Energía $E(v, h)$ para una configuración de neuronas visibles (v) y ocultas (h).
- **Relación con Probabilidad:** La probabilidad de una configuración está dada por la distribución de Boltzmann:

$$P(v, h) = \frac{e^{-E(v, h)}}{Z}$$

Donde Z es una constante de normalización.

- **Significado:** El entrenamiento busca **minimizar la energía** de los estados que corresponden a los datos reales. **Menor energía = Mayor probabilidad**. El sistema "aprende" a asignar baja energía a los patrones de datos que ve frecuentemente.

3. RBM: Algoritmo de Entrenamiento

- **Algoritmo:** El más utilizado es **Contrastive Divergence (CD-k)** (Divergencia Contrastiva).
- **Base:** Debido a que calcular Z (la suma de todas las posibilidades) es computacionalmente imposible, CD aproxima el gradiente.
- **Pasos:**
 1. Toma una muestra de datos (Visible).
 2. Calcula la capa Oculta.
 3. Reconstruye la Visible desde la Oculta.
 4. Vuelve a calcular la Oculta.
 5. Ajusta pesos para acercar la reconstrucción al dato original.

4. Autoencoders: Clasificación de Aprendizaje y Pérdida

- **Tipo de Aprendizaje:** Aunque técnicamente usa algoritmos supervisados (backpropagation), se clasifica como **Aprendizaje No Supervisado** (o auto-supervisado). Esto es porque la "etiqueta" u objetivo es la misma entrada, no una etiqueta externa humana.
- **Funciones de Pérdida:**
 - Si los datos son reales: **Error Cuadrático Medio (MSE)** $\rightarrow ||x - \hat{x}||^2$.
 - Si los datos son binarios/probabilísticos: **Entalpía Cruzada Binaria**.

5. Autoencoders Profundos y RBM (Contexto Histórico)

Esta es la clave del "Renacimiento del Deep Learning" con Geoffrey Hinton (aprox. 2006).

- **El Problema:** Entrenar Autoencoders profundos (muchas capas) solo con Backpropagation fallaba. Los pesos se inicializaban aleatoriamente y el gradiente se desvanecía o quedaba atrapado en mínimos locales malos.
- **La Solución de Hinton (Pre-entrenamiento):**
 - Usó RBMs como bloques de construcción.
 - Entrenaba la primera capa como una RBM (no supervisado). Congelaba los pesos.
 - Usaba la salida de esa capa para entrenar otra RBM encima.
 - Repetía el proceso (**Greedy Layer-wise Pre-training**).
- **Resultado:** Esto inicializaba los pesos de la red profunda en un "lugar óptimo". Finalmente, se aplicaba un "Fine-tuning" con Backpropagation estándar para ajustar toda la red. Esto permitió, por primera vez, entrenar redes profundas de manera efectiva.

Pregunta 5: Diseño de Arquitecturas CNN



Objetivo: Modelar matemáticamente cómo se seleccionan los hiperparámetros (filtros, pooling, profundidad) y analizar sus compromisos (trade-offs).

En el diseño de CNNs, no existe una "fórmula mágica" única que te dé el número exacto, pero sí existen **ecuaciones constitutivas** que restringen el diseño y **leyes de escalamiento** que guían la selección. A continuación, derivamos estas relaciones.

1. Número óptimo de filtros por capa (C_l)

El número de filtros (canales) generalmente aumenta a medida que la red profundiza para compensar la pérdida de resolución espacial (ancho y alto).

- Principio de Diseño:** "Equilibrio de Complejidad Computacional". Se busca que la carga computacional (FLOPs) sea relativamente uniforme entre las capas o bloques.
- Fórmula de Derivación:**
Sea $H_l \times W_l$ la resolución espacial y C_l el número de filtros en la capa l . Si usamos una capa de Pooling que reduce la dimensión espacial a la mitad ($H_{l+1} = H_l/2$), la información espacial se reduce por un factor de 4.
Para preservar la "capacidad de información" o mantener la complejidad constante, a menudo se incrementan los canales por un factor α (usualmente 2).

$$C_{l+1} = \alpha \cdot C_l$$

Generalmente: Si hay *downsampling* $(/2)$, entonces $\alpha = 2$.

Restricción de Memoria:

El número de parámetros (P_l) en una capa convolucional con kernel $K \times K$ es:

$$P_l = (K^2 \cdot C_{l-1} + 1) \cdot C_l$$

La selección de C_l (ancho de la red) debe satisfacer que la suma total sea menor a la memoria disponible:

$$\sum_{l=1}^D P_l \leq \text{Memoria Mxima}$$

2. Configuraci3n de operaciones de Pooling

El pooling determina c3mo se reduce la dimensionalidad.

- F3rmula de Reducci3n de Dimensi3n:**
Para calcular el tama1o de salida (W_{out}) dado un tama1o de entrada (W_{in}), un tama1o de filtro de pooling (F) y un paso o *stride* (S):

$$W_{out} = \left\lfloor \frac{W_{in} - F}{S} \right\rfloor + 1$$

- Criterio de Selecci3n:**
 - El objetivo es ganar **invarianza a la traslaci3n** y reducir el costo.
 - La configuraci3n estndar es **Max Pooling** con $F = 2$ y $S = 2$. Esto reduce la dimensi3n exactamente a la mitad:

$$W_{out} \approx \frac{W_{in}}{2}$$

- Esta configuraci3n descarta el 75% de los datos espaciales, forzando a la red a comprimir la informaci3n en los canales (filtros), justificando el punto 1 (aumentar filtros).

3. Profundidad de la red (D)

La profundidad se selecciona principalmente basndose en el **Campo Receptivo (Receptive Field - RF)** necesario.

- Concepto:** El campo receptivo es la regi3n de la imagen original que "ve" una neurona en la capa l . Queremos que, en la ltima capa, el RF cubra todo el objeto de inter3s.
- F3rmula del Campo Receptivo:**
El RF en la capa l (R_l) se calcula recursivamente:

$$R_l = R_{l-1} + (K_l - 1) \cdot \prod_{i=1}^{l-1} S_i$$

Donde K_l es el tama1o del kernel y S_l el stride de las capas anteriores.

- Derivaci3n de Profundidad Mnima:**
Para clasificar un objeto de tama1o X_{obj} pxeles, necesitamos una profundidad D tal que:

$$R_D \geq X_{obj}$$

Si usamos kernels peque1os (ej. 3×3) para eficiencia, necesitamos una D mayor para alcanzar un RF grande, ya que el RF crece linealmente con la profundidad.

Análisis de Trade-offs y Sensibilidad

El problema de diseño es un problema de optimización con restricciones.

1. Capacidad Representacional vs. Sobreajuste (Overfitting)

- **Capacidad ($\approx$ Complejidad VC):** Crece con el número de parámetros N_{params} .

$$N_{params} \propto D \cdot C^2$$

(Crece linealmente con la profundidad D y cuadráticamente con el ancho C).

- **Análisis:**
 - Si $N_{params} \gg N_{datos}$: **Alto riesgo de Sobreajuste**. La red memoriza.
 - Si $N_{params} \ll$ Complejidad del problema: **Subajuste (Underfitting)**. La red no aprende.
- **Trade-off:** Buscamos la red más profunda (mayor capacidad de abstracción) tal que el error de validación no empiece a subir.

2. Coste Computacional (Sensibilidad)

- La métrica clave son los **FLOPs** (Floating Point Operations). Para una capa convolucional:

$$\text{FLOPs}_l \approx 2 \cdot H_{out} \cdot W_{out} \cdot (K^2 \cdot C_{in} \cdot C_{out})$$

- **Sensibilidad al número de filtros (C):**
Si duplicas los filtros ($2C$), el costo computacional se cuadruplica ($4 \times$ FLOPs), ya que C_{in} y C_{out} se duplican.

$$\text{Sensibilidad: } \frac{\partial \text{Coste}}{\partial C} \propto C$$

- **Sensibilidad a la resolución (H, W):**

El costo es lineal respecto a los píxeles. Por eso se usa Pooling agresivo al inicio para bajar H, W rápidamente y permitir tener muchos filtros C en capas profundas.

Resumen de la "Fórmula" de Selección:

Maximizar la **Precisión (Accuracy)** sujeto a:

1. **Restricción de Cobertura:** Campo Receptivo $\geq$ Tamaño Objeto.
2. **Restricción de Presupuesto:** FLOPs $\leq$ Limite de Hardware.
3. **Restricción de Datos:** N_{params} acorde al tamaño del Dataset (para evitar overfitting).